

Research Architecture Specification

Project: Hybrid Explainable Heterogeneous Graph Anomaly Detection for Healthcare Provider Fraud Detection

Dataset: CMS Healthcare Provider Fraud Detection Analysis (Kaggle / CMS LDS derivative)

Target unit of prediction: Provider (PotentialFraud)

Recommended stack: Heterogeneous graph → R-GCN / GraphSAGE → dual-head (supervised + anomaly) → fusion → GNNExplainer audit

1. System Architecture Diagram Description

Purpose

End-to-end pipeline from raw CMS CSV files to fraud predictions and explainable subgraph reports for auditors.

Layered architecture (top → bottom)

Layer	Name	Responsibility
L1	Data Ingestion Layer	Load Train/Test CSVs (providers, beneficiaries, inpatient/outpatient claims)
L2	Preprocessing & Feature Engineering Layer	Clean, encode, aggregate claim-level statistics to entity-level features
L3	Graph Construction Layer	Build heterogeneous graph $G = (V, E, \tau, \phi)$ with typed nodes and edges
L4	Hybrid Detection Layer	R-GCN encoder → supervised classifier + Isolation Forest on embeddings → score fusion
L5	Explainability Layer	GNNExplainer on flagged provider ego-subgraphs; fidelity scoring
L6	Evaluation & Reporting Layer	CV metrics, baselines, ablations, case-study exports

Publication figure caption (suggested)

Figure 1. System architecture of the proposed hybrid explainable heterogeneous graph anomaly detection framework for Medicare provider fraud. Raw CMS claims are transformed into a heterogeneous provider graph, encoded by an R-GCN, scored by parallel supervised and unsupervised heads, fused into a final fraud risk score, and explained via subgraph attribution for auditor review.

Mermaid — System Architecture



2. Module Descriptions

Module ID	Module Name	Input	Output	Key algorithms	Owner (team)
M1	Data Loader	4 train CSV files	Raw DataFrames	pandas	Student A
M2	Preprocessor	Raw DataFrames	Clean tables + encodings	Label encoding, date parsing, null imputation	Student A
M3	Feature Aggregator	Claim + beneficiary tables	Provider/beneficiary/p hysician feature matrices	GroupBy aggregates, top-K	Student A

Module ID	Module Name	Input	Output	Key algorithms	Owner (team)
				diagnosis/procedure counts	
M4	HeteroGraph Builder	Feature matrices + claim relations	HeteroData (PyG)	Edge list construction, node indexing	Student A
M5	Graph Validator	HeteroData	Schema stats, leakage checks	Provider-disjoint fold validator	Student A
M6	R-GCN Encoder	HeteroData, node features	Provider embeddings $\mathbf{Z} \in \mathbb{R}^{N \times d}$	2-layer HeteroConv + R-GCN	Student B
M7	Supervised Head	$\mathbf{Z}_{\text{provider}}$	$\mathbf{P}_{\text{fraud}} \in [0,1]$	Linear + sigmoid, weighted BCE / focal loss	Student B
M8	Anomaly Scorer	$\mathbf{Z}_{\text{provider}}$ (train non-fraud fit)	$\mathbf{S}_{\text{anomaly}} \in \mathbb{R}$	sklearn Isolation Forest	Student B
M9	Score Fusion	$\mathbf{P}_{\text{fraud}}, \mathbf{S}_{\text{anomaly}}$	$\mathbf{R}_{\text{final}}$	α -blend or logistic stacker (val-tuned)	Student B
M10	Baseline Runner	Tabular provider features + centrality	Baseline metrics	CatBoost, RF, RF+centrality	Student A
M11	CV Orchestrator	Full pipeline	Fold-wise metrics	Stratified 5-fold, provider-disjoint	Student C
M12	Ego-Subgraph Extractor	$\mathbf{G}$, flagged provider IDs	k-hop induced subgraphs	2-hop BFS on hetero projection	Student C

Module ID	Module Name	Input	Output	Key algorithms	Owner (team)
M13	GNNExplainer	Subgraph, trained encoder	Edge/node masks, fidelity	PyG GNNExplainer	Student C
M14	Report Generator	Metrics + explanations	Figures, tables, JSON case studies	matplotlib, export templates	Student C

Module interaction principles

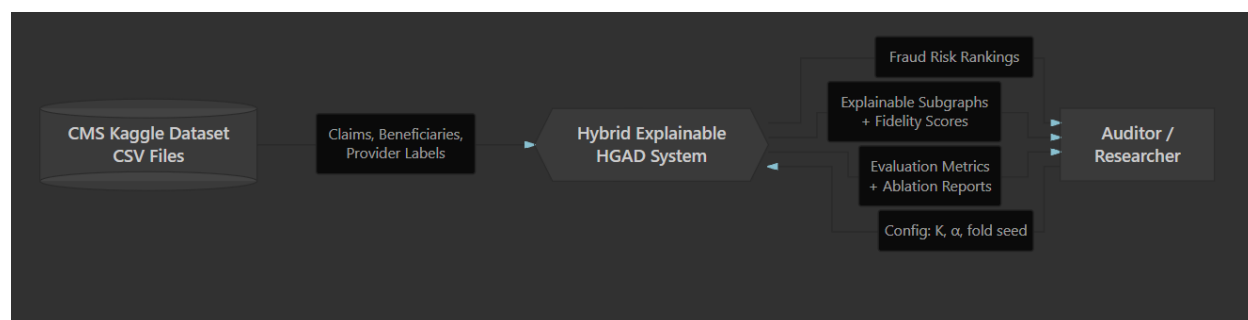
- **M4 → M6** is the only path from raw data to learned representations (no tabular bypass in the main model).
- **M7** and **M8** are **parallel, decoupled heads** sharing encoder output — supports ablation.
- **M13** reads from frozen **M6** weights; no gradient flow during explanation.
- **M10** runs independently for benchmark comparison under **M11**.

3. Data Flow Diagram (DFD)

DFD levels

- **Level 0:** Context diagram (system vs. external entities)
- **Level 1:** Major processing stages

Mermaid — DFD Level 0 (Context)



Mermaid — DFD Level 1



Data store schema (logical)

Store	Contents	Approx. scale (train)
DS1	providers, beneficiaries, physicians, claims (derived)	5.4K / 138K / 100K / 558K rows
DS2	Typed adjacency lists + node feature tensors	~150K–300K edges (coarsened)
DS3	Provider embedding matrix	5,410 × 64
DS4	Per-provider scores + fold assignments	5,410 × 5 folds
DS5	Subgraph masks, fidelity, case metadata	Top-K providers (~50–100)

4. Graph Schema

4.1 Node types

Node type	ID prefix	Count (train, coarsened)	Feature vector (examples)
provider	PRV	5,410	total_reimb, claim_count, n_patients, n_physicians, ip_ratio, avg_claim_amt
beneficiary	BEN	≤138,556 (or top-frequency subset)	age, gender, race, state, chronic_cond_*, annual_reimb
physician	PHY	≤100,737 (or top-frequency subset)	claim_count, n_providers, n_patients, role_distribution
diagnosis	DX	top-500 ICD codes	code frequency, fraud-rate prior (train only)

Coarsening rule (4GB GPU): Include all providers; cap beneficiaries/physicians at top-N by claim frequency (e.g., N=20K each) or aggregate rare entities into <UNK> nodes.

4.2 Edge types (canonical direction)

Edge type	Source → Target	Derivation	Edge attributes
treated	beneficiary → provider	Claim co-occurrence	claim_count, total_reimb
affiliated_with	physician → provider	Attending physician on claim	co_claim_count, total_reimb
diagnosed	provider → diagnosis	Claims billed by provider with diagnosis	frequency, avg_amount
shares_patient	provider → provider	Shared beneficiary (optional)	shared_patient_count, Jaccard

4.3 Target and masks

Target node type: provider

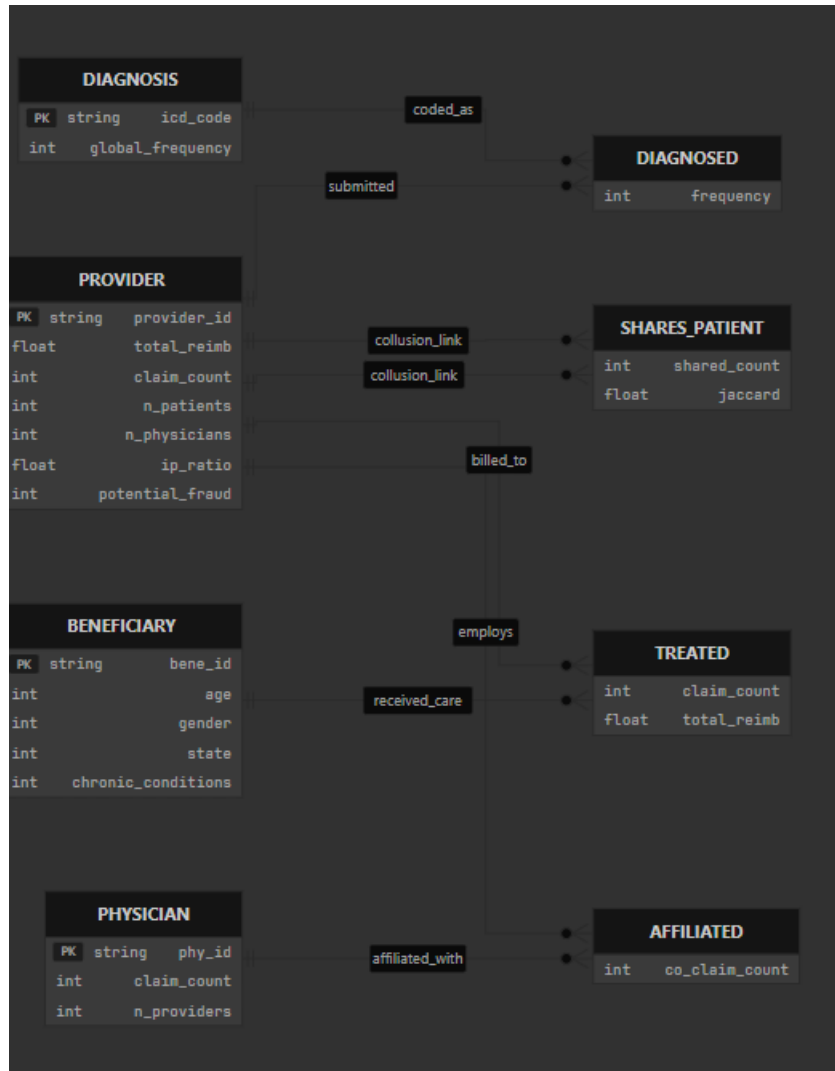
Target label: PotentialFraud $\in \{0, 1\}$

Train mask: provider nodes in train fold

Val mask: provider nodes in validation fold

Test mask: provider nodes in test fold (labels withheld on Kaggle test)

Mermaid — Graph Schema (ER-style)



PlantUML — Graph Schema (for draw.io import)

@startuml GraphSchema

skinparam classAttributeIconSize 0

class Provider <<Node>> {

+provider_id: string

+total_reimb: float

+claim_count: int

+n_patients: int

+potential_fraud: {0,1}

}

class Beneficiary <<Node>> {

+bene_id: string

+age: int

```

+gender: int
+state: int
}
class Physician <<Node>> {
+phy_id: string
+claim_count: int
}
class Diagnosis <<Node>> {
+icd_code: string
+frequency: int
}
Provider "1" -- "0..*" Beneficiary : treated >
Physician "1" -- "0..*" Provider : affiliated_with >
Provider "1" -- "0..*" Diagnosis : diagnosed >
Provider "1" -- "0..*" Provider : shares_patient >
note right of Provider
    **Prediction target**
    Provider-level fraud label
end note
@enduml

```

5. UML Component Diagram

```

@startuml ComponentDiagram
skinparam componentStyle rectangle
package "Data Layer" {
    [Data Loader] as DL
    [Preprocessor] as PP
    [Feature Aggregator] as FA
    [Graph Builder] as GB
}
package "Model Layer" {

```



```

[R-GCN Encoder] as RGCN
[Supervised Head] as SH
[Isolation Forest] as IF
[Score Fusion] as SF
}
package "Explainability Layer" {
  [Ego-Subgraph Extractor] as ESE
  [GNNExplainer] as GEX
  [Fidelity Evaluator] as FE
}
package "Evaluation Layer" {
  [CV Orchestrator] as CV
  [Baseline Runner] as BR
  [Metrics Reporter] as MR
}
interface "CSV Files" as CSV
interface "Fraud Predictions" as PRED
interface "Explanation Reports" as EXP
interface "Metrics JSON" as MET
CSV --> DL
DL --> PP --> FA --> GB
GB --> RGCN
RGCN --> SH
RGCN --> IF
SH --> SF
IF --> SF
SF --> PRED
SF --> ESE --> GEX --> FE --> EXP
SF --> CV
BR --> CV
CV --> MR --> MET

```

note bottom of SF

$$R_{\text{final}} = \alpha \cdot P_{\text{fraud}} + (1-\alpha) \cdot S_{\text{anomaly}}$$

α tuned on validation fold

end note

@enduml

Component dependencies (textual)

Data Loader → Preprocessor → Feature Aggregator → Graph Builder

Graph Builder → R-GCN Encoder

R-GCN Encoder → {Supervised Head, Isolation Forest}

{Supervised Head, Isolation Forest} → Score Fusion

Score Fusion → {Ego-Subgraph Extractor, CV Orchestrator}

Ego-Subgraph Extractor → GNNExplainer → Fidelity Evaluator

Baseline Runner → CV Orchestrator → Metrics Reporter

6. UML Activity Diagram

Process: End-to-end training and inference for one CV fold.

@startuml ActivityDiagram

start

:Load CMS train CSV files;

:Clean and encode features;

:Aggregate claim stats\nper provider/beneficiary/physician;

:Construct heterogeneous graph G;

:Validate provider-disjoint fold split;

partition "Training Phase" {

 :Initialize R-GCN encoder;

 repeat

 :Sample mini-batch neighborhoods;

 :Forward pass → provider embeddings Z;

 :Compute weighted BCE loss\nnon labeled providers;

 :Backpropagate and update R-GCN;

 repeat while (epochs < E_max?) is (yes)

```

->no;

:Extract final provider embeddings  $Z^*$ ;

:Fit Isolation Forest on  $Z^*$  (train non-fraud subset);
}

partition "Inference Phase" {
  :Compute  $P_{\text{fraud}}$  from supervised head;
  :Compute  $S_{\text{anomaly}}$  from IF;
  :Fuse  $R_{\text{final}} = \alpha \cdot P_{\text{fraud}} + (1-\alpha) \cdot S_{\text{anomaly}}$ ;
}

partition "Explainability Phase" {
  if (Provider in top-K flagged?) then (yes)
    :Extract 2-hop ego-subgraph;
    :Run GNNExplainer;
    :Compute explanation fidelity;
    :Export subgraph case study;
  else (no)
  endif
}

:Compute fold metrics (AUPRC, F1, Recall, Precision);

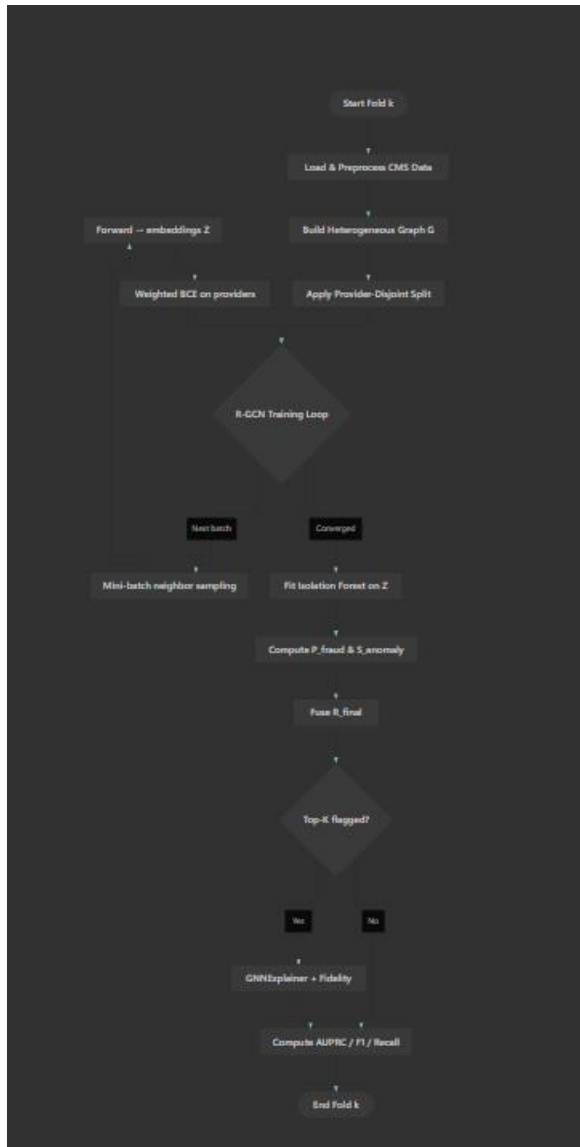
:Store results;

stop

@enduml

```

Mermaid equivalent



7. Sequence Diagram

Scenario: Inference + explanation for a single flagged provider.

@startuml SequenceDiagram

actor Auditor

participant "API / CLI" as CLI

participant "Graph Store" as GS

participant "R-GCN Encoder" as RGCN

participant "Supervised Head" as SH

participant "Isolation Forest" as IF

participant "Score Fusion" as SF

participant "Ego-Subgraph Extractor" as ESE

participant "GNNExplainer" as GEX

Auditor -> CLI: Request fraud assessment\n(provider_id = PRV51003)

CLI -> GS: Fetch provider node +\n2-hop neighborhood

GS --> CLI: Subgraph G_p

CLI -> RGCN: Encode G_p

RGCN --> CLI: Embedding z_p

par Parallel scoring

CLI -> SH: classify(z_p)

SH --> CLI: P_fraud = 0.91

CLI -> IF: score(z_p)

IF --> CLI: S_anomaly = 0.87

end

CLI -> SF: fuse(P_fraud, S_anomaly)

SF --> CLI: R_final = 0.89\n(FRAUD)

CLI -> ESE: extract_ego(G, PRV51003, hops=2)

ESE --> CLI: Subgraph G_p'

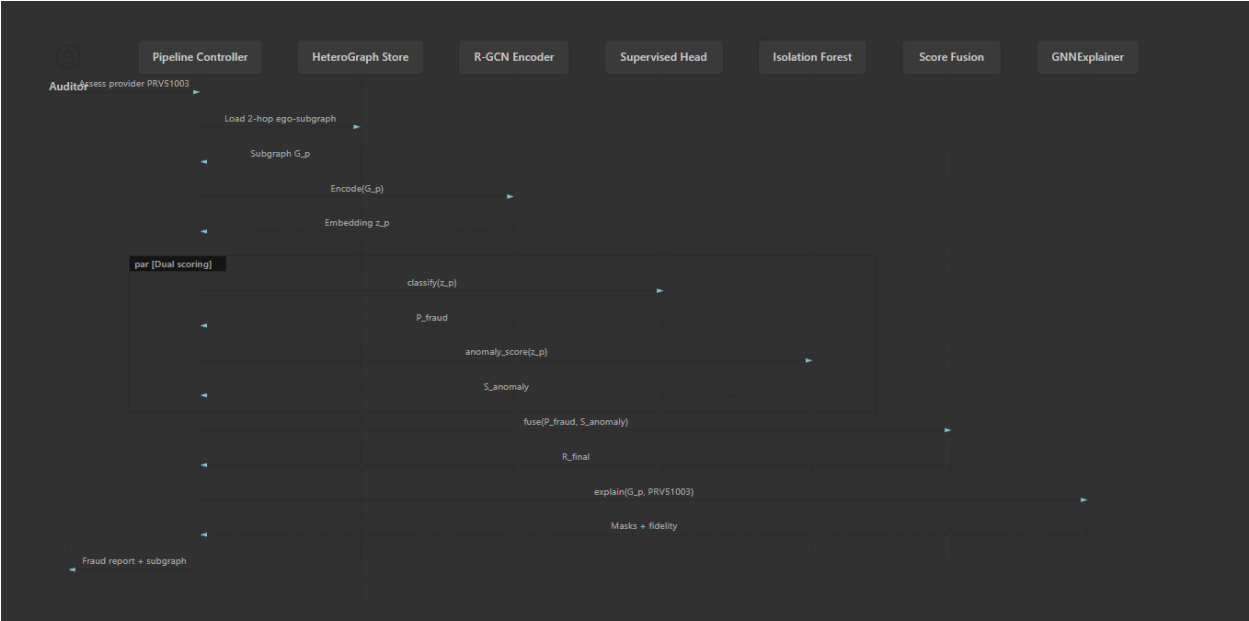
CLI -> GEX: explain(G_p', target=PRV51003)

GEX --> CLI: Edge mask M_e,\nNode mask M_n,\nfidelity = 0.82

CLI --> Auditor: Report:\n- Risk score 0.89\n- Key edges: shared patients,\n physician PHY392922\n- Fidelity 0.82

@enduml

Mermaid equivalent

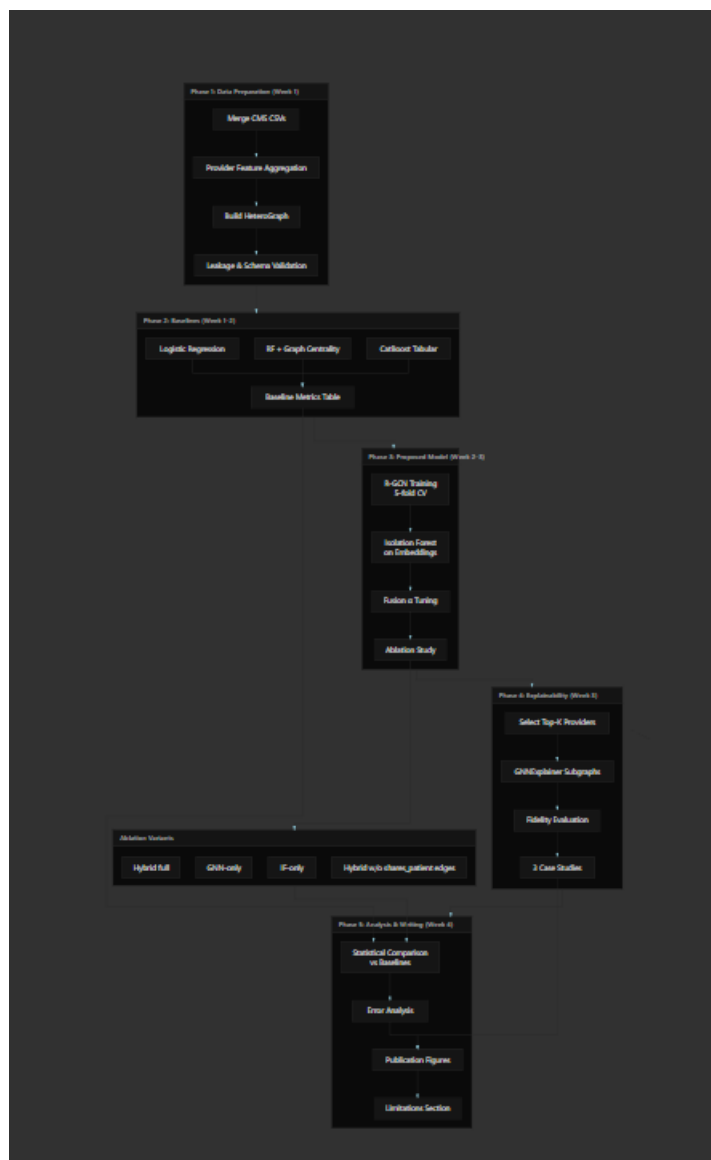


8. Experimental Workflow Diagram

Experimental design summary

Element	Specification
Primary metric	AUPRC (imbalanced provider labels)
Secondary metrics	F1, Recall, Precision, ROC-AUC
CV	5-fold stratified, provider-disjoint
Fusion tuning	$\alpha \in \{0.1,...,0.9\}$ on validation fold only
Explanation set	Top-50 predicted fraud + 10 FP + 10 TN

Mermaid — Experimental Workflow



PlantUML — Experimental Workflow (publication variant)

@startuml ExperimentalWorkflow

|Data Prep|

start

:Merge CMS train files;

:Build heterogeneous provider graph;

:Define 5-fold provider-disjoint CV;

|Baselines|

:Train CatBoost on provider aggregates;

:Train RF + centrality features;

:Record baseline AUPRC / F1;

|Proposed Model|

repeat :For each CV fold k;

:Train R-GCN (weighted BCE);

:Fit IF on train embeddings;

:Tune fusion α on val;

:Evaluate on test fold;

repeat while (k < 5?) is (yes)

->no;

|Ablation|

:Run GNN-only, IF-only, Hybrid;

:Run w/ and w/o provider-provider edges;

|Explainability|

:Select top-50 flagged providers;

:Generate GNNExplainer subgraphs;

:Compute fidelity scores;

:Write 3 auditor case studies;

|Analysis|

:Compare Hybrid vs best baseline\n(McNemar / paired t-test);

:Document limitations\n(label noise, single-year data);

stop

@enduml

Results table structure (for paper Table III)

Model	AUPRC ↑	F1 ↑	Recall ↑	Precision ↑	Train time
Logistic Regression					
CatBoost (tabular)					
RF + Centrality (Yoo baseline)					
R-GCN (supervised only)					

Model	AUPRC ↑	F1 ↑	Recall ↑	Precision ↑	Train time
IF on embeddings only					
Hybrid (R-GCN + IF + fusion)					
Hybrid w/o shares_patient edges					

Draw.io / Export Notes

Diagram	Recommended tool	Export format
System Architecture (§1)	draw.io	PDF/SVG for paper
Data Flow (§3)	draw.io or Mermaid	SVG
Graph Schema (§4)	draw.io ER template	PNG
UML Component (§5)	PlantUML → draw.io plugin	SVG
UML Activity (§6)	PlantUML	SVG
Sequence (§7)	PlantUML or Mermaid	SVG
Experimental Workflow (§8)	draw.io swimlanes	PDF

Color scheme (publication-ready)

- **Data layers:** #E3F2FD (light blue)
- **Model / detection:** #FFF3E0 (light orange)
- **Explainability:** #E8F5E9 (light green)
- **Evaluation:** #F3E5F5 (light purple)
- **Target node (provider):** #FFCDD2 (light red border)

Architecture Decision Records (ADRs)

ADR	Decision	Rationale
ADR-01	Provider-level prediction	Labels are PotentialFraud per provider
ADR-02	Coarsened heterograph	4GB VRAM cannot hold 558K claim nodes

ADR	Decision	Rationale
ADR-03	IF over full GAE	One-month feasibility; IF ablation-compatible
ADR-04	GNNExplainer over SHAP	Graph-native explanations; literature gap
ADR-05	Provider-disjoint CV	Prevents leakage via shared beneficiaries
ADR-06	AUPRC primary metric	9.35% fraud rate; accuracy misleading

This specification is ready for direct import into draw.io (Mermaid plugin), PlantUML renderers, or thesis methodology chapters. If you want, I can next generate a **single consolidated PlantUML file** or a **draw.io XML skeleton** for all eight figures.